

1 · TableGen in 60 seconds

```
class ExampleClass<int parameter> {
  int field = !add(parameter, 1);
  string otherField = ?;
}
def ExampleDef : ExampleClass<5> {
  let otherField = "a value";
}
```

Generators (ODS) look for defs of a known class (e.g. Op); the fields you may let are documented in that class.

TableGen	Literal syntax	C++
int	0 -5 0x5	int64_t
string/code	"C string", [{ raw }]	std::string
bit	0 1 true false	bool
dag	(op Value:\$name, ...)	-
list<T>	[a, b, c]	std::vector
ClassName	ClassName<...>, DefName	ClassName

2 · Skeleton of an Operation

```
class ListOp<string mnemonic, ListTrait> traits = [] {
  Op<ListDialect, mnemonic, traits>;

def ListRangeOp : ListOp<"range", [* traits *]> {
  let summary = "list of a half-open integer range";
  let description = [{ Markdown docs, incl. a fenced mlir example. }];

  let arguments = (ins i32:$lower, i32:$upper);
  let results = (outs ListListType:$result);

  let assemblyFormat = [
    $lower 'to' $upper attr-dict ':' qualified(type($result))
  ];
}

Naming Convention
def name PascalCase; text up to the first _ is dropped from the C++ name →
mlir::List::RangeOp
mnemonic snake_case (sometimes camelCase) → List.range
```

3 · Fields of the Op class

Field	Purpose
arguments	(ins ...) - operands + attributes
results	(outs ...) - result types
regions	(region ...) - attached regions
summary / description	docs (description is Markdown)
assemblyFormat	declarative custom syntax
hasVerifier	= 1 → LogicalResult verify()
hasRegionVerifier	= 1 → verifyRegions()
opDialect, opName, cppNamespace	already set for you by Op<dialect, mnemonic, traits>

4 · TypeConstraints — ins / outs

A TypeConstraint allows one or more types as a result type or as the type of an operand.

TableGen	MLIR	C++ getter type
i1, i8 .. i64	i1, i32	TypedValue<IntegerType>
AnyInteger	any iN	TypedValue<IntegerType>
AnyType	anything	Value
AnyTypeOf<[i32, i64]>	-	Value
List_ListType	!list.list<i32>	TypedValue<ListType>
Optional<X>	0 or 1	Value (null if absent)
Variadic<X>	0 .. n	OperandRange

Extendable via C++ predicates: TypeCPred<":llvm::isa<MyType>(\$self)">, "my type">.

5 · Attributes are arguments too

```
let arguments = (ins List_ListType:$input, // operand
                  i64Attr:$offset, // attribute
                  UnitAttr:$ns, // presence == true
                  OptionalAttr:$i32Attr>:$maybe,
                  DefaultValuedAttr:$i32Attr, "0">:$init);

BoolAttr i32Attr i64Attr IndexAttr i32Attr StrAttr TypeAttr ArrayAttr DenseI64ArrayAttr
SymbolRefAttr AnyAttr, getOffset() returns the unwrapped value (uint64_t), getOffsetAttr()
the IntegerAttr. In assemblyFormat reference it like an operand: $offset.
```

6 · assemblyFormat directives

Describes the syntax after the list.range prefix.

Directive	Meaning
`kw`	literal keyword or punctuation: `, ' `> `< `(`
\$name	operand, attribute, region or successor
type(\$name)	the type of that value
qualified(type(\$x))	force the !dialect. prefix
attr-dict	mandatory – discardable attributes
functional-type(\$in,\$out)	prints (i32, i32) -> i1
custom<Foo>(\$a, type(\$b))	calls C++ parseFoo / printFoo
(\$x^`:` type(\$x))?	optional group, ^ = anchor
regions / successors	all of them at once

Rules: every argument must appear; every type must appear via type(...) unless singleton or deducible; there is always an attr-dict.

7 · Syntax conventions

```
Follow <arguments> attr-dict `:` <types> write types as <arg-types> -> <result-types>.
// DO
%lo = list.push_back %li, %item : !list.list<i32>, i32 -> !list.list<i32>
// DON'T -- types interleaved with the arguments
%lo = list.push_back %li : !list.list<i32>, %item : i32 -> !list.list<i32>
// DON'T -- result type inside the operand type list
%lo = list.push_back %li, %item : !list.list<i32>, i32, !list.list<i32>

Use the bare minimum of type($x) directives; drop -> when one side is deducible or absent:
%r = list.reverse %0 : !list.list<i32>.
```

8 · Type deduction: fewer types in the syntax

Trait	Effect
AllTypesMatch<["a", "b"]>	all named args share one type
SameOperandsAndResultType	everything one type
SameTypeOperands	operands only
TypesMatchWith<...>	arbitrary transform (below)

```
def ListReverseOp : ListOp<"reverse",
  [AllTypesMatch<["input", "result"]>] > {
  let arguments = (ins List_ListType:$input);
  let results = (outs List_ListType:$result);
  // only ONE of input/result must appear in the syntax:
  let assemblyFormat = "$input attr-dict `:` qualified(type($input))";
}

TypesMatchWith<summarv, lhsArg, rhsArg, transform> checks
transform(lhs.getType()) == rhs.getType(); $self is the lhs type. # concatenates strings in
TableGen.

class ListItemTypeMatchesElementType<string listArg, string itemArg>
: TypesMatchWith<"type of " # itemArg # " matches element type",
  listArg, itemArg
  "::llvm::cast<::mlir::List::ListType>($self)"
  ".getElementType()">;
```

9 · Regions

```
let regions = (region AnyRegion:$body);
let regions = (region SizedRegion<1>:$then, SizedRegion<1>:$else);

RegionConstraint Description
AnyRegion unconstrained
SizedRegion<N> exactly N blocks
MinSizedRegion<N> at least N blocks

$body in assemblyFormat prints/parses everything between the braces, block arguments included.
let assemblyFormat = "$input ->` type($result) `with` $body attr-dict";

%mapped = list.map %list -> !list.list<i64> with {
  ^bb0(%elem: i32):
    %ext = arith.extsi %elem : i32 to i64
    list.yield %ext : i64
}
```

10 · Traits used with Regions

Trait	Description
Terminator	marks this op as a terminator
SingleBlockImplicitTerminator< "CppOpClassName">	enforces (and auto-inserts) that terminator
HasParent<"CppOpClassName">	the op owning the enclosing region must be this one
ParentOneOf<[...]>	... one of these ops

```
def SCF_YieldOp : SCF_Op<"yield", [
  Terminator, ParentOneOf<["ForOp", "IfOp", "WhileOp"]>,
]>;
```

11 · Extra verification in C++

Not every invariant fits in TableGen (e.g. "the block argument type must equal the list's element type").

```
let hasVerifier = 1; // runs BEFORE the regions are verified
let hasRegionVerifier = 1; // runs AFTER the regions are verified

// ListOps.cpp
LogicalResult MapOp::verify() {
  Block &body = getBody().front();
  if (body.getNumArguments() != 1)
    return emitOpError("expects the body to have exactly one argument");

  Type elemTy = getInput().getType().getElementType();
  if (body.getArgument(0).getType() != elemTy)
    return emitOpError("expects the body argument type ("
    << body.getArgument(0).getType()
    << ") to match the element type of the list (" << elemTy << ")");
  return success();
}

emitOpError(msg) prefixes 'list.map' op and returns an InFlightDiagnostic you stream into with
<<; success is return success();
```

12 · The generated C++ API

ODS	C++ (ListOps.h.inc / .cpp.inc)
def ListRangeOp	mlir::List::RangeOp
i32:\$lower	getLower(), getLowerMutable()
outs List_ListType:\$result	TypedValue<ListType> getResult()
i64Attr:\$offset	getOffset(), getOffsetAttr(), setOffset()
region AnyRegion:\$body	Region &getBody()

13 · Anatomy of a lit test

Tests live in test/list/*.mlir; every such file is one lit test.

```
// RUN: tutorial-opt --split-input-file %s | FileCheck %s

func.func @push_back(%li: !list.list<i32>, %it: i32) -> !list.list<i32> {
  %l = list.push_back %li, %it : !list.list<i32>
  return %l : !list.list<i32>
}
```

// CHECK-LABEL: func.func @push_back
// CHECK-SAME: (%[[LI:.*]]: !list.list<i32>, %[[IT:.*]]: i32)
// CHECK: %[[L:.*]] = list.push_back %[[LI]], %[[IT]]
// CHECK: return %[[L]]

// RUN: lit runs this as a shell command; the test passes iff it exits with 0. Several RUN: lines = several commands, all must pass.

%s substituted by lit with the path of this file

tutorial-opt parses the IR and prints it again - a roundtrip test proves your assemblyFormat prints what it parses

| FileCheck %s pipes that printed IR into FileCheck, which reads its CHECK directives out of the very same file

--split-input-file cuts the file at every // ----- line and runs tutorial-opt on each piece independently

Run: cmake --build build --target check-tutorial (or check-tutorial-list); one file:
lit -v build/test/list/roundtrip.mlir. By hand:
build/bin/tutorial-opt test/list/roundtrip.mlir.

14 · How FileCheck matches

FileCheck reads the piped-in text **once, top to bottom**. Each directive starts matching where the previous one stopped, so the directives must appear in the same order as their matches in the output.

Directive	Matches
CHECK:	at or after the previous match; unmatched lines in between are skipped
CHECK-NEXT:	on the line immediately after the previous match
CHECK-SAME:	on the same line as the previous match - lets you check one long line with several directives
CHECK-LABEL:	block anchor: the input is split at these and no other directive may match across the boundary. One per function.
CHECK-NOT:	must not occur between the previous and the next matching directive
CHECK-DAG:	a run of CHECK-DAG may match in any order
CHECK-COUNT-3:	the pattern must match 3 times in a row; also CHECK-EMPTY: - "the next line is empty"

Leading and trailing whitespace is ignored and internal whitespace matches any run of blanks, so indent the directives however you like.

15 · FileCheck patterns and captures

Pattern	Meaning
plain text	matched literally, as a substring of the line
{(re)}	regular expression, e.g. {(.*)} for "don't care"
[[NAME:re]]	match re and capture it as variable NAME
[[NAME]]	must equal what NAME captured earlier

SSA value names are renumbered by the printer, so never hard-code them - capture them instead:

```
// CHECK: %[[L:.*]] = list.push_back %{{(.*)}}, %{{(.*)}}
// CHECK: return %[[L]]
```

Idioms: {{(.*)}} - some value, don't care which · {[X:.*]} - bind it · !list.list<{{(.*)}}> - any element type · {[\\[\\]} - a literal [[.

Keep checks loose: assert the structure you care about, not the whole line, otherwise every unrelated change breaks your test.

16 · Testing that verifiers reject IR

```
// RUN: tutorial-opt --split-input-file --verify-diagnostics %s

func.func @two_block_args(%l: !list.list<i32>) {
  // expected-error @below {{expects the body to have exactly one argument}}
  %m = list.map %l -> !list.list<i32> with {
    ^bb0(%a: i32, %b: i32):
      list.yield %a : i32
    }
  return
}
```

With --verify-diagnostics the run passes *iff* the emitted diagnostics match the markers exactly: an unexpected diagnostic fails, and so does an expected-* that never fires. No FileCheck in the pipeline. Put one bad case per // ----- section - the first error usually stops the rest of it.

expected-error also expected-warning, expected-note, expected-remark

{...} a **substring** of the message - keep it short and stable

@below / @above where the diagnostic is: next / previous line, or a relative offset. No marker = same line; @unknown = no location.